

INFORME FINAL — ERSO 2026-007

- [INFORME FINAL — ERSO 2026-007](#)
 - [Resolución y verificación de DID como consumidor conforme](#)
 - [Cómo leer este informe](#)
 - [Contenido](#)
 - [Resumen ejecutivo](#)
- [PARTE I — MARCO CONCEPTUAL: los términos que necesitas](#)
 - [1. La historia en cinco minutos](#)
 - [2. Diccionario de términos](#)
 - [3. Las siete ideas madre](#)
 - [4. El método de trabajo: cómo se piensa un criterio](#)
- [PARTE II — QUÉ PIDE LA ERSO 007, EXPLICADO FRASE POR FRASE](#)
 - [5. La ficha del documento](#)
 - [6. Capacidades del proceso](#)
 - [7. Condiciones del proceso](#)
 - [8. Descripción del proceso](#)
 - [9. Qué debe hacer: los cinco pasos](#)
 - [10. Los cuatro criterios de aceptación](#)
- [PARTE III — EL PROYECTO: QUÉ SE CONSTRUYÓ PARA ESTA ERSO Y CÓMO ESTÁ MONTADO](#)
 - [11. Visión general](#)
 - [12. El entorno para esta ERSO](#)
 - [13. Mapa de requisitos a implementación](#)
- [PARTE IV — LABORATORIOS DE DEMOSTRACIÓN: LA PRUEBA MATERIAL DE CADA PASO Y CADA CRITERIO](#)
 - [14. Cómo funcionan los laboratorios](#)
 - [15. Preparación común](#)
 - [16. Laboratorio A — Pasos 1 y 2: resolver y comprobar la conformidad](#)
 - [17. Laboratorio B \(parte 1\) — La prueba por dentro](#)
 - [18. CRITERIO 1 — Una prueba firmada por la clave resuelta se valida correctamente](#)
 - [19. CRITERIO 2 — Una prueba firmada por una clave ajena se rechaza](#)
 - [20. CRITERIO 3 — Un documento no conforme, con id desajustado o inexistente, se rechaza](#)
 - [21. CRITERIO 4 — El consumidor no realiza operaciones de escritura ni de publicación](#)
 - [22. Pruebas automáticas y recorrido de extremo a extremo](#)
- [PARTE V — PREGUNTAS Y RESPUESTAS](#)
 - [23. Preguntas sobre los conceptos](#)
 - [24. Preguntas por laboratorio y por criterio](#)
- [PARTE VI — CÓMO RESPONDER A CADA CRITERIO](#)
 - [25. Redacción modelo de las cuatro respuestas](#)
 - [26. Lista de comprobación para quien acepta](#)
- [PARTE VII — LÍMITES, HALLAZGOS Y OPERACIÓN](#)
 - [27. Lo que este informe NO demuestra, y lo que se encontró](#)
 - [28. Si algo no sale como en el informe](#)
 - [29. Operación diaria](#)
- [ANEXOS](#)
 - [Anexo A — Los scripts](#)
 - [Anexo B — Autocomprobación \(con respuestas\)](#)
 - [Anexo C — Ejercicios del método \(con respuesta modelo\)](#)

INFORME FINAL — ERSO 2026-007

Resolución y verificación de DID como consumidor conforme

Manual de comprensión, construcción y comprobación

Proyecto	Identidad Digital Soberana SSI
ERSo	2026-007 — Resolución y verificación DID como consumidor conforme
Desarrollador asignado (según la ERSo)	Geovani Rincón
Plantilla de pruebas y pruebas funcionales (según la ERSo)	Luis González
Fechas de la ERSo	Elaboración: viernes 18-sep-2026 · Desarrollo: viernes 25-sep-2026, 7:30 a. m. · Prueba: miércoles 30-sep-2026, 7:30 a. m.
Dominio del laboratorio	civica-desarrollo.avance.org.co
Módulo	did-resolver (no depende del servicio de registro)
Ubicación del proyecto	/home/geovani/Descargas/generic/bitacora/new_proyect
Salidas de terminal de este informe	Obtenidas ejecutando los comandos reales sobre el proyecto, el 1 de octubre de 2026
Complementos	scripts/lab-007-completo.sh (todos los laboratorios en un script) · informes/ERSo-2026-007.md (versión corta) · informes finales de las ERSo 004, 005 y 006

Nota de fechas. La fecha de prueba que fija la ERSo (30-sep-2026) es anterior a la fecha de este informe (1-oct-2026). Este documento se puede usar tanto para preparar la repetición de la prueba como para dejar constancia de ella; las firmas de la tabla de actividades son de las personas responsables.

Cómo leer este informe

Este documento está escrito para que **cualquier persona** pueda entender qué se pidió, qué se construyó y cómo se comprobó, **sin necesidad de ser experta**. Cada término técnico se explica la primera vez que aparece; la Parte I es un diccionario de esos términos.

Símbolo	Significa
💡	Idea clave: lo más importante de la sección, en una frase
🔍	Qué vas a ver: lo que debes esperar <i>antes</i> de ejecutar un comando
✅	Qué significa: cómo interpretar lo que el sistema respondió

Símbolo	Significa
	Cuidado: una trampa frecuente o un límite que conviene conocer
	Predice antes de ejecutar: una pregunta para que pienses antes de ver el resultado

Si eres...	Lee esto
Quien decide o aprueba	El Resumen ejecutivo y la Parte VII
Quien necesita entender el problema	Parte I y Parte II
Quien debe reproducir y comprobar	Partes III y IV
Quien prepara la prueba funcional	Partes IV, VI y la lista de comprobación (§26)
Quien prepara una exposición	Partes V y VI

Cómo están presentados los comandos. Los bloques `console` son una **sesión de terminal real**: las líneas con \$ son lo que se escribe; las demás, lo que el sistema respondió. Los valores que cambian en cada ejecución (horas, identificadores, claves, huellas) serán distintos al repetir; lo que debe coincidir es el **patrón** explicado debajo de cada salida.

Esta ERSO es distinta de las anteriores. Las ERSO 004 a 006 construyen y operan el **productor** (quien publica DID Documents). Esta construye el **consumidor** (quien los lee y decide si confía). Es un código separado, que ni siquiera sabe escribir en el registro.

Contenido

Parte	Secciones	De qué trata
Resumen ejecutivo	—	El problema, la solución y el resultado en una página
I · Marco conceptual	1 a 4	Los términos, en lenguaje sencillo, y el método de trabajo
II · Qué pide la ERSO	5 a 10	Capacidades, condiciones, descripción, pasos y criterios, frase por frase
III · El proyecto	11 a 13	Qué se construyó, cómo se monta y cómo funciona por dentro
IV · Laboratorios	14 a 22	La prueba material de cada paso y criterio, con cada comando y su respuesta
V · Preguntas y respuestas	23 y 24	Todas las preguntas de comprensión con su respuesta
VI · Cómo responder	25 y 26	Redacción modelo de cada criterio y lista de comprobación
VII · Límites y operación	27 a 29	Lo que NO se demuestra, qué hacer si algo falla y operación diaria
Anexos	A a D	Scripts, autocomprobación, ejercicios y referencias

Resumen ejecutivo

El problema. Publicar un DID Document no sirve de nada si quien lo lee no sabe **comprobar** que es válido. Imagina que alguien presenta una credencial firmada «por Avance». El verificador

debe: encontrar la clave pública de Avance, asegurarse de que esa clave es realmente de Avance y está autorizada para firmar ese tipo de cosa, y recién entonces comprobar la firma. Un verificador descuidado podría aceptar un documento mal formado, uno de otra entidad, o una firma con una clave ajena.

La solución. Un **consumidor conforme**: un componente de solo lectura que, dado un DID, (1) lo **resuelve** (obtiene el documento por HTTPS), (2) comprueba que es **estructuralmente conforme** con el modelo de datos de W3C, (3) **verifica** una prueba (firma) contra la clave publicada y autorizada para el propósito pedido, y (4) **rechaza** todo lo que no cumpla: documentos defectuosos, DID inexistentes o desactivados, y claves ajenas. Además **no puede escribir**: ni su interfaz ni sus dependencias lo permiten.

El resultado.

#	Criterio	Resultado	Dónde se ve
1	Una prueba firmada por la clave resuelta se valida	✓	§18
2	Una prueba firmada por una clave ajena se rechaza	✓	§19
3	Un documento no conforme, con <code>id</code> desajustado o inexistente, se rechaza	✓	§20
4	El consumidor no realiza operaciones de escritura ni de publicación	✓	§21

Cifras: 12 pruebas unitarias del resolvedor + 1 prueba de interoperabilidad con el registro real, todas en verde; 12 casos de documentos defectuosos servidos por un servidor malicioso de laboratorio, y el servidor registró **solo peticiones GET**.

Lo que debes saber de antemano: (1) el consumidor valida contra el documento **actual**: una firma hecha con una clave ya rotada se rechaza (§27); (2) solo acepta claves `Multikey` P-256; (3) no incluye caché (§27).

PARTE I — MARCO CONCEPTUAL: los términos que necesitas

1. La historia en cinco minutos

Un portero de edificio recibe una carta con un sello que dice «Notaría Avance». Antes de dejarla pasar, hace cinco preguntas:

1. ¿Existe esa notaría en el directorio oficial? (**¿el DID existe?**)
2. ¿La hoja del directorio está bien hecha, y es de esa notaría y no de otra? (**¿el documento es conforme y su id coincide?**)
3. ¿Ese sello figura en la hoja? (**¿la clave existe?**)
4. ¿Está autorizado para este tipo de carta? (**¿la clave está autorizada para ese propósito?**)
5. ¿El sello de la carta coincide con el del directorio? (**¿la firma es válida?**)

Si **cualquiera** falla, la carta no pasa. Y el portero **nunca escribe** en el directorio: solo lo consulta.

💡 Un consumidor conforme es ese portero: **desconfía por defecto** y deja pasar solo lo que supera todas las comprobaciones.

2. Diccionario de términos

2.1 Los de siempre (resumen)

Término	En palabras sencillas
DID	Identificador con la forma <code>did:web:dominio:ruta</code> . Es una <i>dirección</i> que se puede calcular.
<code>did:web</code>	Método de DID donde el identificador se convierte en una URL: <code>did:web:ejemplo.org:entidades:avance</code> → <code>https://ejemplo.org/entidades/avance/did.json</code> .
DID Document	El JSON público de un DID: su <code>id</code> , sus claves públicas y para qué sirven.
Clave pública / privada	La privada firma y no se comparte; la pública verifica.
P-256 / ES256	La curva de las claves y el algoritmo de firma usados (ECDSA con SHA-256).
Multikey	Formato estándar para escribir una clave pública como texto (<code>zDn...</code>).
Hash (<code>sha256:...</code>)	Huella digital de un contenido.
VDR	El registro que guarda y entrega los DID Document (ERSo 004).

2.2 Resolver y verificar

Término	En palabras sencillas
Consumidor	Quien <i>usa</i> un DID para confiar en algo. No publica.
Productor	Quien <i>publica</i> DID Documents (ERSo 004 a 006).
Resolver (<i>resolve</i>)	Pasar del DID al documento completo . «Dame el documento de ...:avance».
Dereferenciar (<i>dereference</i>)	Pasar de una DID URL con fragmento (...:avance#key-1) a un recurso concreto dentro del documento (esa clave).
DID URL	Un DID más una parte extra: did:web:...:avance#key-1. El #key-1 se llama <i>fragmento</i> .
DID Resolution	La especificación (W3C) que define cómo se resuelve y qué errores se devuelven.
Metadatos de resolución	Información <i>sobre</i> el proceso: si hubo error, tipo de contenido, URL leída.
Metadatos del documento	Información <i>sobre el documento</i> : si está desactivado, su versión (<i>versionId</i>), su hash (<i>contentHash</i>).
Consumidor conforme	Un cliente que sigue las reglas del estándar y rechaza lo que no las cumple.
Conformidad estructural	Que el documento tenga la forma que exige el modelo de datos: id válido, contexto correcto, métodos de verificación bien formados, referencias que no «cuelgan».
Perfil CONSUMER / PUBLISHER	Dos conjuntos de reglas del validador. El del productor es estricto y propio (lista blanca); el del consumidor es el que exige el estándar , más tolerante con propiedades extra pero igual de severo con lo grave (secretos, <i>id</i> , claves).

2.3 Pruebas y relaciones de verificación

Término	En palabras sencillas
Prueba (proof)	Una afirmación firmada. Aquí, un JWS compacto.
JWS (compacto)	Texto con tres partes separadas por puntos: <i>cabecera.contenido.firma</i> . La cabecera dice el algoritmo y el <i>kid</i> .
<i>kid</i>	Identificador de la clave que firmó: aquí una DID URL (<i>did:web:...#key-1</i>).
Relación de verificación	«Para qué» sirve una clave en un documento. Las dos que trata esta ERSo:
— <i>authentication</i>	La clave sirve para demostrar quién eres (iniciar sesión, responder un desafío).
— <i>assertionMethod</i>	La clave sirve para afirmar cosas (firmar credenciales, emitir afirmaciones).
Clave autorizada	Una clave que aparece en la relación correcta. Existir en <i>verificationMethod</i> no basta : debe estar listada en la relación pedida.
Firma inválida	La firma no corresponde a la clave publicada (otra clave o contenido alterado).

2.4 Defensas del cliente

Término	En palabras sencillas
Solo lectura	El componente no tiene ninguna operación que cambie algo en el mundo: solo <i>GET</i> .

Término	En palabras sencillas
No seguir redirecciones	Si el servidor responde «mira en otro sitio», el consumidor no obedece : podría ser un desvío hacia un servidor del atacante.
Límite de tamaño	Rechazar documentos enormes (aquí 128 KiB) para no agotar memoria.
Tipo de contenido (Content-Type)	Lo que el servidor dice que entrega. Si no es JSON/DID, se rechaza.
Tiempo de espera	Si el servidor no responde en pocos segundos, se abandona.
Código HTTP 410	«Gone»: el recurso existió y fue retirado. En DID significa desactivado .

3. Las siete ideas madre

1. **Resolver no es confiar**: obtener el documento es solo el primer paso.
2. **Verificar es una cadena**: si un eslabón falla, se rechaza.
3. **Existir no es estar autorizado**: la clave debe estar en la relación adecuada.
4. **El id manda**: el documento debe llamarse como se pidió.
5. **Desconfiar de la red**: sin redirecciones, con límites de tamaño y de tiempo.
6. **El consumidor no escribe**: ni por interfaz ni por dependencias.
7. **Se valida contra el documento actual**: lo que se rota, deja de valer.

4. El método de trabajo: cómo se piensa un criterio

1. **Descomponer** el criterio en el verbo y la evidencia que pide.
 2. **Hacerlo comprobable**: ¿qué comando lo demuestra?
 3. **Buscar el caso negativo**: aquí casi todo son «debe rechazarse».
 4. **Predecir** el resultado antes de ejecutar.
 5. **Ejecutar y observar**.
 6. **Correlacionar** con la regla del código que lo causa.
 7. **Redactar** la respuesta con la evidencia.
-

PARTE II — QUÉ PIDE LA ERSO 007, EXPLICADO FRASE POR FRASE

5. La ficha del documento

Campo del PDF	Valor
Proyecto	Identidad Digital Soberana SSI
Título	Resolución y verificación DID como consumidor conforme
Desarrollador	Geovani Rincón
Fecha de la ERSO	Viernes, 18 de septiembre de 2026
Fecha de desarrollo	Viernes, 25 de septiembre de 2026, 7:30 a. m.
Fecha de prueba	Miércoles, 30 de septiembre de 2026, 7:30 a. m.
Responsables	Análisis y diseño: Karen Flórez Madieto · Asignar desarrollador, aceptar archivos y archivo: Rocío Villamizar · Implementar: Geovani Rincón · Plantilla de pruebas y pruebas funcionales: Luis González

⚠ Las firmas y fechas de la tabla de actividades **corresponden a las personas responsables**; este informe aporta las **evidencias técnicas**, no las firmas.

Capacidades → Condiciones → Descripción → Qué debe hacer (5 pasos) → Criterios (4) → Actividades

6. Capacidades del proceso

6.1 La frase principal, desarmada

"Se requiere habilitar **de forma opcional** la **resolución y verificación** de DID Documents **como consumidor**, contra las **versiones normativas fijadas**, **sin alterar el camino base del piloto**."

#	Fragmento	En lenguaje sencillo	Cómo se materializó
1	"de forma opcional"	Es un módulo aparte: quien no lo use, no lo carga	Módulo <code>did-resolver</code>
2	"resolución y verificación"	Dos acciones: obtener el documento y comprobar pruebas	<code>DidWebResolver</code> + <code>ProofVerifier</code>

#	Fragmento	En lenguaje sencillo	Cómo se materializó
3	"como consumidor"	Lee; no publica	Interfaz sin operaciones de escritura
4	"versiones normativas fijadas"	Se sigue una versión concreta de cada estándar	docs/VERSIONES-NORMATIVAS.md
5	"sin alterar el camino base"	Nada de lo anterior se rompe	Módulo independiente; e2e.sh completo en verde

6.2 Las cuatro capacidades

#	Capacidad (texto del PDF)	Qué se busca	Cómo se resolvió	Paso
1	Resolver un DID mediante el cliente de resolución del consumidor	Pasar de DID a documento	DidWebResolver.resolve	1
2	Recuperar el documento y comprobar su conformidad estructural con el modelo W3C	No aceptar documentos mal formados	DidDocumentValidator (perfil CONSUMER)	2
3	Validar relaciones y pruebas (authentication, assertionMethod) con la clave Multikey resuelta	Que la firma y la autorización de la clave sean correctas	ProofVerifier.verify	3
4	Rechazar documentos no conformes, DID inexistentes o claves no autorizadas	Fallar de forma segura y explicada	Códigos de error normativos y propios	4

7. Condiciones del proceso

Condición 1 — "El método resoluble es `did:web`; no se integra ION ni anclajes equivalentes."

- **Qué significa.** Solo se sabe resolver `did:web`. Otros métodos (que guardan sus documentos en cadenas de bloques) no están soportados.
- **Qué problema evita.** Complejidad y dependencias innecesarias.
- **Cómo se cumple.** Todo DID que no sea `did:web` devuelve `invalidDid`.

Condición 2 — "La resolución es una operación del consumidor, no un requisito de publicación del productor."

- **Qué significa.** Para *publicar* no hace falta *resolver*, y viceversa; son responsabilidades distintas.
- **Qué problema evita.** Acoplar los dos lados: si el productor dependiera del consumidor, un fallo del resolutor impediría publicar.
- **Cómo se cumple.** `vdr-service` solo usa `did-resolver` en sus **pruebas** (`testImplementation`), no en su código de producción (§12).

Condición 3 — "El consumidor no realiza operaciones de escritura ni de publicación."

- **Qué significa.** Solo lee.
- **Qué problema evita.** Que un componente expuesto a documentos ajenos (potencialmente hostiles) tenga poder de modificar algo.
- **Cómo se cumple.** Interfaz con dos operaciones de lectura; ninguna llamada de escritura en el código; ninguna dependencia del servicio de registro (§21).

8. Descripción del proceso

"Quien consume un DID necesita obtener el documento asociado y comprobar que es válido antes de confiar en una prueba."

El orden importa: **primero** validar el documento, **después** confiar en la prueba.

*"Conviene distinguir **resolver** (obtener el DID Document a partir del DID) de **dereferenciar** (seguir una URL concreta dentro del documento)."*

Dos operaciones distintas, ambas implementadas (§11).

"Este componente actúa como consumidor conforme: obtiene el documento, valida su estructura y sus relaciones de verificación, y rechaza lo que no cumpla."

Tres verbos: **obtener, validar, rechazar.**

"No emite juicios sobre identidad jurídica ni sobre atributos civiles."

Importante: que una firma sea válida significa «esta clave firmó», **no** «esta persona es quien dice ser ante la ley». Es un límite deliberado (§27).

"Literatura y temas a consultar"

Lectura	Qué te aporta
W3C DIDs v1.1	El modelo de datos y las relaciones de verificación
W3C DID Resolution v1	Definición de «resolver» frente a «dereferenciar» y los códigos de error
Método <code>did:web</code>	La regla DID → URL
Buenas prácticas de caché y control de versión en resolutores	Por qué conviene exponer <code>versionId</code> y <code>contentHash</code>

9. Qué debe hacer: los cinco pasos

1. Resolver el DID mediante el cliente de resolución del consumidor.
2. Recuperar el DID Document y comprobar su conformidad estructural con el modelo de datos de W3C DIDs.

3. Validar las relaciones y pruebas (authentication, assertionMethod) con la clave pública Multikey resuelta.
4. Rechazar documentos no conformes, con DID inexistente o con claves no autorizadas.
5. Asegurar que el consumidor opera en modo solo lectura, sin escritura ni publicación.

Paso	Lo que pide	Criterio que lo comprueba	Laboratorio
1	Resolver	Base de todos	\$16
2	Conformidad estructural	Criterio 3	\$16 y \$20
3	Validar relaciones y pruebas	Criterios 1 y 2	\$18 y \$19
4	Rechazar	Criterios 2 y 3	\$19 y \$20
5	Solo lectura	Criterio 4	\$21

10. Los cuatro criterios de aceptación

Criterio 1. "Una prueba firmada por la clave resuelta se valida correctamente; evidencia: prueba positiva." **Criterio 2.** "Una prueba firmada por una clave ajena se rechaza; evidencia: prueba negativa." **Criterio 3.** "Un documento no conforme, con id desajustado o inexistente, se rechaza; evidencia: prueba negativa." **Criterio 4.** "El consumidor no realiza operaciones de escritura ni de publicación; evidencia: revisión de interfaz y pruebas de solo lectura."

Criterio	Verbo y evidencia	Lo que debo poder mostrar	El caso contrario	Mecanismo
1	Se valida · prueba positiva	Una firma legítima da <code>VALIDA</code>	—	Resolución + autorización + verificación de firma
2	Se rechaza · prueba negativa	Firma con otra clave, alterada o con clave inexistente → rechazo	Cada variante da un código distinto	<code>INVALID_SIGNATURE</code> , <code>KEY_NOT_FOUND</code> , <code>KEY_NOT_AUTHORIZED</code> ...
3	Se rechaza · prueba negativa	<code>id</code> desajustado, DID inexistente, desactivado, documento defectuoso → rechazo	12 variantes de documento	Validador <code>CONSUMER</code> + defensas del cliente
4	No realiza · revisión de interfaz y pruebas	Interfaz sin escritura, sin dependencia, sin llamadas	Un servidor espía solo ve <code>GET</code>	Diseño del módulo

⚠ **Observación sobre el criterio 3.** Reúne tres cosas distintas en una frase: documento **no conforme**, con `id` **desajustado** y **inexistente**. Cada una tiene su propio código y su propia prueba (§20).

PARTE III — EL PROYECTO: QUÉ SE CONSTRUYÓ PARA ESTA ERSO Y CÓMO ESTÁ MONTADO

11. Visión general

11.1 Qué se construyó

Un **módulo de código** llamado `did-resolver`, con dos clases principales (224 líneas en total):

Clase	Archivo	Qué hace
<code>DidWebResolver</code>	<code>Resolver.kt</code>	Resuelve (<code>resolve</code>) y dereferencia (<code>dereference</code>)
<code>ProofVerifier</code>	<code>ProofVerifier.kt</code>	Verifica una prueba JWS contra la clave resuelta
<code>VerificationKeyResolver</code>	<code>ProofVerifier.kt</code>	Obtiene solo la clave pública de una DID URL para un propósito (lo usa el servicio de credenciales)
<code>DidResolver</code> (interfaz)	<code>Resolver.kt</code>	El contrato de solo lectura

```
$ wc -l did-resolver/src/main/kotlin/co/org/avance/ssi/resolver/*.kt
 86 did-resolver/src/main/kotlin/co/org/avance/ssi/resolver/ProofVerifier.kt
138 did-resolver/src/main/kotlin/co/org/avance/ssi/resolver/Resolver.kt
224 total
```

11.2 Cómo se usa (y cómo está montado en Docker)

A diferencia del registro (ERSo 004), el consumidor **no es un servicio con su propio contenedor**: es una **biblioteca** que otros componentes incorporan. El proyecto está organizado en módulos de Gradle:

```
$ sed -n '/^include(/,/)/p' settings.gradle.kts
include(
    "did-core", "did-resolver", "vdr-service", "did-tools",
    // ERSO 2026-001/002/003 (cartera, credenciales, DID del titular)
    "credentials-core", "wallet-core", "wallet-sim", "wallet-service", "credential-service",
)
$ grep -n 'project(":did-resolver")' */build.gradle.kts | cut -c1-120
credential-service/build.gradle.kts:13:    implementation(project(":did-resolver"))
did-tools/build.gradle.kts:13:    implementation(project(":did-resolver"))
vdr-service/build.gradle.kts:25:    testImplementation(project(":did-resolver"))
wallet-service/build.gradle.kts:26:    testImplementation(project(":vdr-service"))
wallet-sim/build.gradle.kts:14:    implementation(project(":did-resolver"))
```

✓ Cómo leerlo:

- `did-resolver` lo usan `credential-service` (para localizar la clave del emisor al verificar credenciales), `did-tools` (la caja de herramientas) y `wallet-sim` (la cartera simulada).

- `vdr-service` (el registro) **solo lo usa en pruebas** (`testImplementation`): su código de producción no lo necesita. Esto materializa la condición 2.
- `did-resolver` **no depende** de `vdr-service`: es independiente.

En Docker, el consumidor se ejecuta dentro del contenedor `tools`, que lleva incorporada la herramienta `did-tools` (y con ella `did-resolver`):

```
$ sed -n '/^  tools:\/,\/depends_on:/p' deploy/docker-compose.yml; sed -n '/depends_on:/,+/p'
deploy/docker-compose.yml | sed -n '1,2p' >/dev/null
tools:
  build:
    context: ..
    dockerfile: deploy/Dockerfile
    args: { APP: did-tools }
  image: vdr-ssi/did-tools:local
  profiles: ["tools"]
  user: "${LOCAL_UID:-1000}:${LOCAL_GID:-1000}"
  volumes:
    - ./certs:/certs:ro
    - ../evidencias/work:/work
  working_dir: /work
  depends_on:
```

- `profiles: ["tools"]` — el contenedor **no arranca** con el resto; se lanza bajo demanda con `docker compose --profile tools run`.
- `./certs:/certs:ro` — monta los certificados **en solo lectura** (aquí, para confiar en nuestra CA).
- `../evidencias/work:/work` — carpeta de trabajo donde se leen/escriben las claves y documentos de prueba.
- Corre en la **misma red de Docker** que el resto: por eso puede resolver nombres como `civica-desarrollo.avance.org.co` (que apunta a `nginx`) y `evil.lab` (el servidor malicioso de laboratorio, §20).

11.3 El recorrido de una verificación

```
prueba JWS → 1 ¿está bien formada? — no → MALFORMED_PROOF
(cabecera.contenido.firma) |
    ↓
    2 ¿el DID del kid es el esperado? — no → DID_MISMATCH
    ↓
    3 resolver el DID (GET https://.../did.json)
      | no existe → RESOLUTION_FAILED (notFound)
      | desactivado (410) → DEACTIVATED
      | documento no conforme → RESOLUTION_FAILED (invalidDidDocument...)
      ↓
    4 ¿la clave (kid) existe en el documento? — no → KEY_NOT_FOUND
      ↓
    5 ¿está autorizada para el propósito? — no → KEY_NOT_AUTHORIZED
      ↓
    6 ¿la clave es una Multikey P-256 válida? — no → INVALID_KEY
      ↓
    7 ¿la firma corresponde a esa clave? — no → INVALID_SIGNATURE
      ↓
    VALIDA
```

💡 Cada cuadrito es una **puerta**; todas deben abrirse. Las ocho respuestas negativas posibles tienen **código propio**: eso permite que quien use el módulo sepa *por qué* se rechazó.

12. El entorno para esta ERSO

12.1 Preparar el terminal

Se usa el mismo entorno de las ERSO anteriores, más tres atajos específicos:

Atajo	Qué hace
ESCRIBIR	Publica un documento (rol de productor); se usa para preparar el DID de prueba
VERIFICAR	Verifica una prueba (rol de consumidor) con nuestra CA de confianza
RESOLVER	Resuelve un DID y muestra solo los metadatos

⚠ Fíjate en la separación: **ESCRIBIR** es el único atajo que **escribe** y pertenece al productor; los otros dos son el consumidor. Para esta ERSO, el productor solo sirve para tener **algo que consumir**.

```
$ cd /home/geovani/Descargas/generic/bitacora/new_proyect/deploy
$ set -a; . ./env; set +a; export LOCAL_UID=$(id -u) LOCAL_GID=$(id -g)
$ D=$VDR_DOMAIN
$ W=./evidencias/work
$ PUB="--cacert certs/ca.crt --resolve $D:8443:127.0.0.1"
$ TOOLS() { docker compose --progress quiet --profile tools run --rm -T tools "$@"; }
$ ESCRIBIR() { local cid=$1 sec=$2 pl2=$3 did=$4; shift 4; TOOLS write --admin-url "https://$D:8443"
--client-id "$cid" --secret "$sec" --p12 "/certs/$pl2.p12" --p12-pass "$P12_PASS" --ca /certs/ca.crt
--did "$did" "$@"; }
$ VERIFICAR() { local jws=$1; shift; TOOLS verify --jws "$jws" --ca /certs/ca.crt "$@"; }
$ RESOLVER() { TOOLS resolve --did "$1" --ca /certs/ca.crt 2>&1 | grep -E
"^(resolutionMetadata|documentMetadata)" | cut -c1-260; }
```

12.2 Comprobar que el registro está vivo

```
$ curl -s $PUB https://$D:8443/health
{"status":"UP","vdr":"enabled"}
```

12.3 Un identificador de prueba único

```
$ ID=inst-007-$(date +%s); DID=did:web:$D:entidades:$ID; echo "DID=$DID"
DID=did:web:civica-desarrollo.avance.org.co:entidades:inst-007-1790874467
```

13. Mapa de requisitos a implementación

Requisito de la ERSO	Implementación	Dónde
Paso 1: resolver	DidWebResolver.resolve: DID → URL https → GET	Resolver.kt
Paso 2: conformidad estructural	DidDocumentValidator.validate(..., Profile.CONSUMER)	did-core
Paso 3: validar relaciones y pruebas	ProofVerifier.verify + isAuthorized	ProofVerifier.kt

Requisito de la ERSo	Implementación	Dónde
Paso 4: rechazar	Códigos <code>invalidDid</code> , <code>notFound</code> , <code>invalidDidDocument</code> , <code>representationNotSupported</code> , <code>invalidDidDocumentLength</code> , <code>internalError</code> ; y <code>MALFORMED_PROOF</code> , <code>DID_MISMATCH</code> , <code>RESOLUTION_FAILED</code> , <code>DEACTIVATED</code> , <code>KEY_NOT_FOUND</code> , <code>KEY_NOT_AUTHORIZED</code> , <code>INVALID_KEY</code> , <code>INVALID_SIGNATURE</code>	<code>Resolver.kt</code> , <code>ProofVerifier.kt</code>
Paso 5: solo lectura	Interfaz <code>DidResolver</code> ; sin dependencia de <code>vdr-service</code>	<code>Resolver.kt</code> , <code>build.gradle.kts</code>
Defensas del cliente	Sin redirecciones; Content-Type acotado; 128 KiB; 5 s	<code>Resolver.kt</code>

Las **defensas del cliente** merecen detalle porque son las que se prueban en el criterio 3:

Defensa	Valor	Por qué
Redirecciones	No se siguen	Un servidor hostil podría desviar hacia otro sitio
Content-Type aceptado	<code>application/did+json</code> , <code>application/did+ld+json</code> , <code>application/json</code>	Rechazar HTML u otros formatos
Tamaño máximo	128 KiB	Evitar agotar memoria
Tiempo de espera	5 s (3 s para conectar)	No colgarse
Esquema	Siempre <code>https</code> (salvo hosts «inseguros» explícitos de laboratorio)	No leer por canal sin cifrar

PARTE IV — LABORATORIOS DE DEMOSTRACIÓN: LA PRUEBA MATERIAL DE CADA PASO Y CADA CRITERIO

14. Cómo funcionan los laboratorios

Cada laboratorio:  **predicción** → **comandos** (cada uno con su respuesta real y su explicación: qué hace, qué se busca, qué significa) →  **conclusión**.

Lab	Prueba...	Sección
A	Pasos 1 y 2 — resolver y comprobar la conformidad	\$16
B	Paso 3 — la prueba por dentro, y criterio 1	\$17 y \$18
C	Criterio 2 — pruebas con clave ajena y otras falsas	\$19
D	Criterio 3 — documentos no conformes, <code>id</code> desajustado, inexistentes, desactivados	\$20
E	Criterio 4 — solo lectura	\$21
G	Pruebas automáticas	\$22

 Es **una sola sesión continua**: A y B preparan lo que usan C a E. El script `scripts/lab-007-completo.sh` la ejecuta entera, incluida la limpieza final.

15. Preparación común

La preparación (terminal, atajos y DID de prueba) está en \$12.1 a \$12.3. Para esta ERSo el DID de prueba es `...:entidades:inst-007-<hora>`.

16. Laboratorio A — Pasos 1 y 2: resolver y comprobar la conformidad

Qué se busca. Publicar un documento (con el rol de productor, solo para tener algo que leer) y comprobar que el **consumidor** lo resuelve por HTTPS, lo valida y entrega metadatos.

Predice antes de ejecutar

Al resolver, el cliente imprime el documento y dos líneas de metadatos. ¿Qué esperas en `error`, en `violations` y en `versionId`? ¿De dónde saca el cliente el `versionId`?

16.1 Preparar el documento a consumir

Se genera una clave, se construye el documento (con el servicio de emisión) y se publica con las credenciales de Avance:

```
$ T00LS keygen --out c007-clave.json; T00LS build-doc --did $DID --key c007-clave.json --service-url "https://$D/issuer" --out c007-doc.json >/dev/null; echo "(documento de prueba construido: $(jq -c keys $W/c007-doc.json))"
Clave P-256 generada. Multikey público: zDnaefaQ9gg6F16KULHDpU4JBbAU8mYKb1EvdFYPzHocvu7Uw
(documento de prueba construido:
["@context","assertionMethod","authentication","id","service","verificationMethod"])
$ ESCRIBIR avance-issuer $CLIENT_SECRET_AVANCE avance-issuer $DID --purpose CREATE --expected 0 --key c007-clave.json --doc c007-doc.json 2>&1 | grep -E "^[4/4\]|\"status\|\"version\""
[4/4] escritura .... 201 If-Match=0 Idempotency-Key=0f26c372-2048-4721-b53e-de148fda1c7a
"status": "CONFIRMED",
"version": 1,
```

- **Qué hace.** `keygen` crea el par de claves; `build-doc` arma el documento; `ESCRIBIR ... CREATE` lo publica (el filtro `grep` deja solo el estado final).
- **Qué significa.** `201` y `CONFIRMED`, versión 1. A partir de aquí, el documento existe en la web y el productor ya no interviene.

16.2 El DID se convierte en una dirección

```
$ T00LS did --domain $D --namespace entidades/$ID; curl -s -o /dev/null -w "GET directo a esa dirección (puerto 8443 del laboratorio) -> HTTP %{http_code}\n" $PUB
"https://$D:8443/entidades/$ID/did.json"
did:web:civica-desarrollo.avance.org.co:entidades:inst-007-1790874467
https://civica-desarrollo.avance.org.co/entidades/inst-007-1790874467/did.json
GET directo a esa dirección (puerto 8443 del laboratorio) -> HTTP 200
```

- **Qué hace.** `tools did` aplica la regla de `did:web` (primera línea: el DID; segunda: la URL). Luego se pide esa URL con `curl`.
- **Qué significa.** El consumidor no «busca» el documento: **calcula** su dirección a partir del DID. Responde `200`.

16.3 Resolver con el cliente del consumidor

```
$ T00LS resolve --did $DID --ca /certs/ca.crt
didDocument: {
  "@context": [
    "https://www.w3.org/ns/did/v1",
    "https://w3id.org/security/multikey/v1"
  ],
  "assertionMethod": [
    "did:web:civica-desarrollo.avance.org.co:entidades:inst-007-1790874467#key-1"
  ],
  "authentication": [
    "did:web:civica-desarrollo.avance.org.co:entidades:inst-007-1790874467#key-1"
  ],
  "id": "did:web:civica-desarrollo.avance.org.co:entidades:inst-007-1790874467",
  "service": [
    {
      "id": "did:web:civica-desarrollo.avance.org.co:entidades:inst-007-1790874467#issuer",
      "serviceEndpoint": "https://civica-desarrollo.avance.org.co/issuer",
      "type": "OID4VCI"
    }
  ],
}
```

```

    "verificationMethod": [
      {
        "controller": "did:web:civica-desarrollo.avance.org.co:entidades:inst-007-1790874467",
        "id": "did:web:civica-desarrollo.avance.org.co:entidades:inst-007-1790874467#key-1",
        "publicKeyMultibase": "zDnaefaQ9gg6F16kULHDpU4JBbAU8mYKb1EvdFYPzHocvu7Uw",
        "type": "Multikey"
      }
    ]
  }
}
resolutionMetadata: ResolutionMetadata(error=null, message=null, contentType=application/did+json,
url=https://civica-desarrollo.avance.org.co/entidades/inst-007-1790874467/did.json, violations=[])
documentMetadata: DocumentMetadata(deactivated=false, versionId=1,
contentHash=sha256:053ac80d519aae6012e03529184d0ea53e2b76839100ab3d2adb5b19268c3e60)

```

- **Qué hace.** `resolve` ejecuta `DidWebResolver.resolve`: calcula la URL, hace `GET` por HTTPS (validando el certificado con la CA), comprueba el tipo de contenido y el tamaño, valida la estructura (perfil `CONSUMER`) y entrega el resultado.
- **Qué se busca.** `error=null` y `violations=[]` (aceptado), más metadatos coherentes.
- **Qué significa.**
 - `contentType=application/did+json`: el tipo correcto.
 - `url=...`: la dirección realmente leída.
 - `versionId=1`: proviene del encabezado `ETag` del servidor.
 - `contentHash=sha256:...`: el hash **que calcula el propio consumidor** del contenido recibido (no se fía del que dice el servidor).

🔗 Conclusión del laboratorio A

El consumidor resuelve un `did:web` calculando su URL, lo lee solo por HTTPS, valida su forma y devuelve documento + metadatos.

17. Laboratorio B (parte 1) — La prueba por dentro

Antes de verificar, conviene ver **qué es** una prueba.

17.1 Firmar un mensaje

```

$ JWS=$(TOOLS sign --key c007-clave.json --kid "$DID#key-1" --message "solicitud-de-presentacion" |
tail -1); echo "${JWS:0:70}..."
eyJJhbGciOiJIJFuzI1NiIsImtpZCI6ImRpZDp3ZWl6Y2l2aWNhLWRLc2Fycm9sbG8uYXZhbml...
```

- **Qué hace.** `sign` firma el mensaje `solicitud-de-presentacion` con la **clave privada** del archivo, declarando que la clave es `<DID>#key-1`.
- **Qué significa.** Resulta un texto largo con **tres partes separadas por puntos**.

17.2 Abrir la prueba

```

$ dec() { local s=$(echo "$1" | tr "_-" "/+"); while [ $(( ${#s} % 4 )) -ne 0 ]; do s="$s="; done;
echo "$s" | base64 -d 2>/dev/null; }; echo "cabecera : $(dec $(echo $JWS | cut -d. -f1))"; echo
"contenido: $(dec $(echo $JWS | cut -d. -f2))"; echo "firma      : $(echo $JWS | cut -d. -f3 | cut -c1-
40)..." (64 bytes, ES256)"
cabecera : {"alg":"ES256","kid":"did:web:civica-desarrollo.avance.org.co:entidades:inst-007-
1790874467#key-1"}
contenido: solicitud-de-presentacion
firma      : IOB5X-iGnyjlVW7f6ahMmca0LMHqkCTN8I8RbBG-... (64 bytes, ES256)
```

- **Cabecera:** dice el algoritmo (ES256) y el `kid`, **la clave que dice haber firmado**.
- **Contenido:** el mensaje.
- **Firma:** 64 bytes producidos con la clave privada. Solo quien tiene esa clave pudo producirla.

💡 Observa que **la propia prueba dice qué clave la firmó**, pero eso es solo una afirmación. El trabajo del consumidor es comprobar si es cierto.

18. CRITERIO 1 — Una prueba firmada por la clave resuelta se valida correctamente

Criterio. "Una prueba firmada por la clave resuelta se valida correctamente; evidencia: prueba positiva."

🔧 Predice antes de ejecutar

Hay una sola clave en el documento, listada en `authentication` y en `assertionMethod`. Si verifico la misma prueba para cada propósito, ¿qué debería pasar?

18.1 Verificar con el propósito «asertar»

```
$ VERIFICAR "$JWS" --purpose assertionMethod --did $DID
VALIDA did=did:web:civica-desarrollo.avance.org.co:entidades:inst-007-1790874467 kid=did:web:civica-desarrollo.avance.org.co:entidades:inst-007-1790874467#key-1 payload=solicitud-de-presentacion
```

- **Qué hace.** `verify` ejecuta toda la cadena (§11.3): lee el `kid`, resuelve el DID, localiza la clave, comprueba que esté en `assertionMethod`, decodifica el Multikey y verifica la firma. `-did` indica el DID que **se esperaba** (si la prueba dijera otro, se rechazaría).
- **Qué significa.** `VALIDA` con el DID, la clave y el mensaje recuperado.

18.2 Verificar con el propósito «autenticar»

```
$ VERIFICAR "$JWS" --purpose authentication --did $DID
VALIDA did=did:web:civica-desarrollo.avance.org.co:entidades:inst-007-1790874467 kid=did:web:civica-desarrollo.avance.org.co:entidades:inst-007-1790874467#key-1 payload=solicitud-de-presentacion
```

Igual de válida: la clave está también en `authentication`.

18.3 Con el DID institucional real de Avance

```
$ JWSA=$(TOOLS sign --key avance.json --kid "did:web:$D:entidades:avance#key-1" --message "solicitud-de-presentacion" | tail -1); VERIFICAR "$JWSA" --purpose assertionMethod --did did:web:$D:entidades:avance
VALIDA did=did:web:civica-desarrollo.avance.org.co:entidades:avance kid=did:web:civica-desarrollo.avance.org.co:entidades:avance#key-1 payload=solicitud-de-presentacion
```

Se repite sobre `entidades/avance` con la clave real de la institución: `VALIDA`. Es el caso de uso de verdad: **comprobar una firma de Avance con solo su DID**.

Conclusión — evidencia del criterio 1

La prueba firmada por la clave publicada se valida para cada propósito autorizado, tanto en un DID de prueba como en el real.

19. CRITERIO 2 — Una prueba firmada por una clave ajena se rechaza

Criterio. "Una prueba firmada por una clave ajena se rechaza; evidencia: prueba negativa."

Predice antes de ejecutar

Un atacante fabrica una prueba que dice `kid = <DID de Avance>#key-1`, pero la firma con **su** clave. ¿Qué código esperas? ¿Y si en lugar de cambiar la clave, alguien altera el mensaje de una prueba legítima?

19.1 Una clave ajena

```
$ T00LS keygen --out c007-ajena.json; JWSX=$(T00LS sign --key c007-ajena.json --kid "$DID#key-1" --message "solicitud-de-presentacion" | tail -1); VERIFICAR "$JWSX" --purpose assertionMethod --did $DID
Clave P-256 generada. Multikey público: zDnaetXs9xeAwGddzmgenkiY1zKPrqQtTR8rfLXFHqukGnPAv
RECHAZADA INVALID_SIGNATURE: La firma no corresponde a la clave publicada
```

- **Qué hace.** Genera otra clave (la del «atacante») y firma con ella afirmando ser `#key-1` del DID de prueba.
- **Qué significa.** `RECHAZADA INVALID_SIGNATURE`. El consumidor tomó la clave **publicada** (no la del atacante) y la firma no corresponde. Nótese que el rechazo ocurre en el **último** eslabón: la prueba estaba bien formada, el DID existe, la clave existe y está autorizada; solo la **firma** es falsa.

19.2 Contenido alterado

```
$ H=$(echo $JWS | cut -d. -f1); S=$(echo $JWS | cut -d. -f3); P2=$(printf "solicitud-ALTERADA" | base64 -w0 | tr "+/" "-_" | tr -d "="); VERIFICAR "$H.$P2.$S" --purpose assertionMethod --did $DID
RECHAZADA INVALID_SIGNATURE: La firma no corresponde a la clave publicada
```

- **Qué hace.** Toma la prueba legítima y sustituye el contenido (`solicitud-ALTERADA`), conservando cabecera y firma.
- **Qué significa.** También `INVALID_SIGNATURE`: la firma protege el contenido; cambiar un carácter la invalida.

19.3 Una clave que no existe

```
$ JWSK=$(T00LS sign --key c007-clave.json --kid "$DID#key-9" --message "x" | tail -1); VERIFICAR "$JWSK" --purpose assertionMethod --did $DID
RECHAZADA KEY_NOT_FOUND: La clave did:web:civica-desarrollo.avance.org.co:entidades:inst-007-1790874467#key-9 no existe en el documento
```

La prueba declara `#key-9`, que no existe: `KEY_NOT_FOUND`. Se rechaza **antes** de mirar la firma.

19.4 Un DID que no es el esperado

```
$ VERIFICAR "$JWS" --purpose assertionMethod --did did:web:$D:entidades:avance
RECHAZADA DID_MISMATCH: La prueba fue firmada por did:web:civica-
desarrollo.avance.org.co:entidades:inst-007-1790874467 y se esperaba did:web:civica-
desarrollo.avance.org.co:entidades:avance
```

La prueba es perfectamente válida, pero es de **otro DID** distinto del esperado: `DID_MISMATCH`. Es la defensa contra aceptar una prueba legítima de otra entidad en un contexto equivocado.

19.5 Pruebas mal formadas

```
$ VERIFICAR "esto-no-es-una-prueba" --purpose assertionMethod; VERIFICAR "a.b.c" --purpose
assertionMethod
RECHAZADA MALFORMED_PROOF: JWS compacto mal formado

RECHAZADA MALFORMED_PROOF: JWS mal formado: Input byte[] should at least have 2 bytes for base64
bytes
```

Dos pruebas basura: ambas dan `MALFORMED_PROOF`. El consumidor ni siquiera intenta resolver.

19.6 La rotación de claves

Si el productor cambia la clave del DID (versión 2), lo firmado con la clave **vieja** deja de valer:

```
$ TOOLS build-doc --did $DID --key c007-ajena.json --service-url "https://$D/issuer" --out c007-doc-
b.json >/dev/null; ESCRIBIR avance-issuer $CLIENT_SECRET_AVANCE avance-issuer $DID --purpose UPDATE -
-expected 1 --key c007-clave.json --doc c007-doc-b.json 2>&1 | grep -E "^\\
[4/4\\]|\\\"status\\\"|\\\"version\\\""
[4/4] escritura .... 200 If-Match=1 Idempotency-Key=18368c6e-5ec8-45d4-adf8-929f498b3c64
  "status": "CONFIRMED",
  "version": 2,
$ echo "-- la firma de ANTES (clave vieja) --"; VERIFICAR "$JWS" --purpose assertionMethod --did
$DID; echo "-- una firma NUEVA (clave nueva) --"; JWSN=$(TOOLS sign --key c007-ajena.json --kid
"$DID#key-1" --message "solicitud-de-presentacion" | tail -1); VERIFICAR "$JWSN" --purpose
assertionMethod --did $DID
-- la firma de ANTES (clave vieja) --
RECHAZADA INVALID_SIGNATURE: La firma no corresponde a la clave publicada

-- una firma NUEVA (clave nueva) --
VALIDA did=did:web:civica-desarrollo.avance.org.co:entidades:inst-007-1790874467 kid=did:web:civica-
desarrollo.avance.org.co:entidades:inst-007-1790874467#key-1 payload=solicitud-de-presentacion
```

- **Qué hace.** `UPDATE` con un documento que lleva la clave nueva (la prueba de posesión la firma la clave **vieja**, que es la vigente). Luego se verifica la firma de antes y una nueva.
- **Qué significa.** La firma vieja da `INVALID_SIGNATURE`; la nueva, `VALIDA`. El consumidor valida contra el documento **actual**. (Consecuencia importante en §27.)

Conclusión — evidencia del criterio 2

Variante	Código
Clave ajena	<code>INVALID_SIGNATURE</code>
Contenido alterado	<code>INVALID_SIGNATURE</code>

Variante	Código
Clave inexistente	KEY_NOT_FOUND
DID distinto al esperado	DID_MISMATCH
Prueba mal formada	MALFORMED_PROOF
Clave retirada por rotación	INVALID_SIGNATURE

20. CRITERIO 3 — Un documento no conforme, con id desajustado o inexistente, se rechaza

Criterio. "Un documento no conforme, con id desajustado o inexistente, se rechaza; evidencia: prueba negativa."

Qué se busca. Rechazo de tres familias: (a) DID **inexistente** (o desactivado), (b) documento con **id desajustado**, (c) documento **no conforme** por otras razones.

Predice antes de ejecutar

Un servidor entrega un documento JSON perfecto... pero con "id" de otro DID. ¿El consumidor debe aceptarlo? ¿Por qué sería peligroso aceptarlo?

20.1 Un DID inexistente

```
$ DIDF=did:web:$D:entidades:fantasma-$ID; RESOLVER $DIDF; JWSF=$(TOOLS sign --key c007-clave.json --kid "$DIDF#key-1" --message "x" | tail -1); VERIFICAR "$JWSF" --purpose assertionMethod
resolutionMetadata: ResolutionMetadata(error=notFound, message=El DID no existe en https://civica-desarrollo.avance.org.co/entidades/fantasma-inst-007-1790874467/did.json, contentType=null, url=https://civica-desarrollo.avance.org.co/entidades/fantasma-inst-007-1790874467/did.json, documentMetadata: DocumentMetadata(deactivated=false, versionId=null, contentHash=null)
RECHAZADA RESOLUTION_FAILED: notFound: El DID no existe en https://civica-desarrollo.avance.org.co/entidades/fantasma-inst-007-1790874467/did.json
```

- **Qué hace.** Resuelve un DID que nunca se publicó y luego intenta verificar una prueba a su nombre.
- **Qué significa.** El servidor responde **404** y el consumidor traduce a **notFound** (código normativo). Al verificar, se convierte en **RESOLUTION_FAILED**.

20.2 Un DID desactivado

Se publica un DID y se **desactiva**; el servidor pasa a responder **410 Gone**.

```
$ DIDB=did:web:$D:entidades:baja-$ID; T00LS keygen --out c007-baja.json >/dev/null; T00LS build-doc -did $DIDB --key c007-baja.json --out c007-baja-doc.json >/dev/null; ESCRIBIR avance-issuer $CLIENT_SECRET_AVANCE avance-issuer $DIDB --purpose CREATE --expected 0 --key c007-baja.json --doc c007-baja-doc.json 2>&1 | grep -E "^[4/4]|\\"status\\""; echo "-- ahora se desactiva --"; ESCRIBIR avance-issuer $CLIENT_SECRET_AVANCE avance-issuer $DIDB --purpose DEACTIVATE --expected 1 --key c007-baja.json 2>&1 | grep -E "^[4/4]|\\"status\\"|\\"purpose\\""
[4/4] escritura .... 201 If-Match=0 Idempotency-Key=a545e5f7-a29d-4e78-b680-f0b818a08150
"status": "CONFIRMED",
-- ahora se desactiva --
```

```
[4/4] escritura ... 200 If-Match=1 Idempotency-Key=561866a8-6b2c-46f5-9df7-27fa72336585
  "purpose": "DEACTIVATE",
  "status": "CONFIRMED",
$ RESOLVER $DIDB; JWSB=$(TOOLS sign --key c007-baja.json --kid "$DIDB#key-1" --message "x" | tail
-1); VERIFICAR "$JWSB" --purpose assertionMethod
resolutionMetadata: ResolutionMetadata(error=null, message=DID desactivado, contentType=null,
url=https://civica-desarrollo.avance.org.co/entidades/baja-inst-007-1790874467/did.json, violations=
[])
documentMetadata: DocumentMetadata(deactivated=true, versionId=null, contentHash=null)
RECHAZADA DEACTIVATED: El DID está desactivado
```

- **Qué significa.** Para un DID desactivado el consumidor **no devuelve error de resolución** (`error=null`) sino que marca `deactivated=true` y **no entrega documento**. Al verificar una prueba, el resultado es `DEACTIVATED`. Una clave de una entidad dada de baja **no vale**.

20.3 Un servidor malicioso de laboratorio

Hasta ahora, los documentos eran los del registro, que **ya filtra** lo que se publica. Para probar cómo se defiende el consumidor de documentos defectuosos —que un registro bien portado jamás entregaría— hace falta un servidor que **no filtre**. Se monta uno propio, en un contenedor `evil-lab`.

Paso 1. Un certificado para `evil.lab`, firmado por nuestra CA (para que el cliente confíe en la conexión, pero no en el contenido):

```
$ EV=$W/evil007; rm -rf $EV; mkdir -p $EV/www/casos;
$ openssl req -new -newkey ec -pkeyopt ec_paramgen_curve:prime256v1 -nodes -keyout $EV/evil.key -subj
"/CN=evil.lab" -out $EV/evil.csr 2>/dev/null
$ printf "subjectAltName=DNS:evil.lab\n" > $EV/san.cnf
$ openssl x509 -req -in $EV/evil.csr -CA certs/ca.crt -CAkey certs/ca.key -set_serial 0x1007 -days 30
-extfile $EV/san.cnf -out $EV/evil.crt 2>&l | grep -v "^$"
$ openssl x509 -in $EV/evil.crt -noout -subject -issuer -ext subjectAltName
Certificate request self-signature ok
subject=CN = evil.lab
subject=CN = evil.lab
issuer=CN = VDR Laboratorio CA, O = Avance
X509v3 Subject Alternative Name:
  DNS:evil.lab
```

Paso 2. Los documentos defectuosos. Se parte de un documento bueno y se le hace una alteración distinta por caso:

```
$ EV=$W/evil007
$ TOOLS build-doc --did did:web:evil.lab:casos:bueno --key c007-clave.json --out evil007/base.json
>/dev/null
$ mk() { mkdir -p $EV/www/casos/$1; sed "s/casos:bueno/casos:$1/g" $EV/base.json | jq "$2" >
$EV/www/casos/$1/did.json; }
$ mk bueno '.'
$ mk idajeno '.id="did:web:evil.lab:casos:otro"'
$ mk sinctx 'del(."@context")'
$ mk privada '.verificationMethod[0].privateKeyMultibase="z1234"'
$ mk dangling '.assertionMethod=["did:web:evil.lab:casos:dangling#key-9"]'
$ mk tipo '.verificationMethod[0].publicKeyMultibase="zDnaeNoEsUnaClaveValida"'
$ mk soloauth '.assertionMethod=[] | .id="did:web:evil.lab:casos:soloauth" |
.verificationMethod[0].id="did:web:evil.lab:casos:soloauth#key-1" |
.verificationMethod[0].controller="did:web:evil.lab:casos:soloauth" | .authentication=
["did:web:evil.lab:casos:soloauth#key-1"]'
$ mk grande '.relleno=("x"*200000)'
$ mk html '.'
$ mkdir -p $EV/www/casos/notjson; echo "esto no es json" > $EV/www/casos/notjson/did.json
```



```
$ ls -l $EV/www/casos | tr "\n" " "; echo; wc -c $EV/www/casos/bueno/did.json
$EV/www/casos/grande/did.json
bueno dangling grande html idajeno notjson privada sinctx soloauth tipo
 532 ../evidencias/work/evil007/www/casos/bueno/did.json
200554 ../evidencias/work/evil007/www/casos/grande/did.json
201086 total
```

Caso	Alteración	Qué defecto representa
bueno	Ninguna	Control: debe aceptarse
idajeno	id de otro DID	id desajustado
sinctx	Sin @context	Falta el contexto DID
privada	Campo privateKeyMultibase	Material privado
dangling	assertionMethod apunta a una clave que no existe	Referencia colgante
tipo	Clave pública no decodificable	Clave inválida
notjson	Texto que no es JSON	Contenido ilegible
html	Documento bueno, pero con Content-Type: text/html	Tipo de contenido ajeno
grande	Documento de 200 KB	Excede el límite
redir	Respuesta 302 hacia el documento bueno	Redirección
gone	Respuesta 410	Desactivado
noexiste	Ruta sin archivo (404)	Inexistente
soloauth	Clave solo en authentication	Para la prueba de §20.5

Paso 3. Poner el servidor en marcha (con la configuración de las respuestas especiales):

```
$ EV=$W/evil007
> cat > $EV/evil.conf <<'EOF'
> server {
>     listen 443 ssl;
>     server_name evil.lab;
>     ssl_certificate /etc/nginx/evil.crt;
>     ssl_certificate_key /etc/nginx/evil.key;
>     root /usr/share/nginx/html;
>     location = /casos/redir/did.json { return 302 https://evil.lab/casos/bueno/did.json; }
>     location = /casos/gone/did.json { return 410; }
>     location = /casos/html/did.json { types { } default_type text/html; }
> }
> EOF
> docker rm -f evil-lab >/dev/null 2>&1
> docker run -d --name evil-lab --network vdr-ssi_default --network-alias evil.lab -v "$(cd $EV &&
pwd)/evil.conf:/etc/nginx/conf.d/default.conf:ro" -v "$(cd $EV &&
pwd)/evil.crt:/etc/nginx/evil.crt:ro" -v "$(cd $EV && pwd)/evil.key:/etc/nginx/evil.key:ro" -v "$(cd
$EV && pwd)/www:/usr/share/nginx/html:ro" nginx:1.27-alpine >/dev/null; sleep 2; docker ps --filter
name=evil-lab --format "table {{.Names}}\t{{.Status}}"
NAMES      STATUS
evil-lab   Up 2 seconds
```

- **Qué hace.** `docker run` lanza un nginx en la **red de Docker del proyecto**, con el nombre de red `evil.lab` (`--network-alias`), el certificado, la configuración (las respuestas `302` , `410` y el tipo `text/html`) y los documentos montados **solo lectura**.
- **Qué significa.** Un servidor que responde `Up` . El consumidor lo encontrará en `https://evil.lab/casos/<caso>/did.json` .

20.4 Resolver los doce casos

```
$ for c in bueno idajeno sinctx privada dangling tipo notjson html grande redir gone noexiste; do
printf "==" %9s " $c; TOOLS resolve --did did:web:evil.lab:casos:$c --ca /certs/ca.crt 2>&1 | grep
"^resolutionMetadata" | sed -E "s/.*error=([^\,]*)).*violations=(\[.*\])\).*error=\1
violaciones=\2/"; done
== bueno      error=null violaciones=[]
== idajeno    error=invalidDidDocument violaciones=[ID_MISMATCH, INVALID_VERIFICATION_METHOD]
== sinctx     error=invalidDidDocument violaciones=[INVALID_CONTEXT]
== privada    error=invalidDidDocument violaciones=[PRIVATE_KEY_MATERIAL]
== dangling   error=invalidDidDocument violaciones=[DANGLING_REFERENCE]
== tipo       error=invalidDidDocument violaciones=[INVALID_KEY]
== notjson    error=invalidDidDocument violaciones=[]
== html       error=representationNotSupported violaciones=[]
== grande     error=invalidDidDocumentLength violaciones=[]
== redir      error=internalError violaciones=[]
== gone       error=null violaciones=[]
== noexiste   error=notFound violaciones=[]
```

✓ **Cómo leerlo.** Cada línea es un caso y lo que el consumidor concluyó:

Caso	Resultado	Por qué
bueno	error=null	Control: lo acepta
idajeno	invalidDidDocument · ID_MISMATCH (+ INVALID_VERIFICATION_METHOD)	El id no coincide con el DID pedido
sinctx	invalidDidDocument · INVALID_CONTEXT	Falta el contexto
privada	invalidDidDocument · PRIVATE_KEY_MATERIAL	Trae clave privada
dangling	invalidDidDocument · DANGLING_REFERENCE	Referencia a algo que no existe
tipo	invalidDidDocument · INVALID_KEY	La clave no se puede decodificar
notjson	invalidDidDocument	No es un objeto JSON
html	representationNotSupported	Tipo de contenido no aceptado
grande	invalidDidDocumentLength	Más de 128 KiB
redir	internalError («Redirecciones no permitidas»)	No obedece desvíos
gone	error=null, deactivated=true	Se interpreta como desactivado
noexiste	notFound	No existe

💡 `idajeno` es la respuesta a la predicción: aceptar un documento con un `id` ajeno permitiría que un atacante te sirva el documento de **otra entidad** y lo hagas pasar por el pedido. Es el rechazo que nombra literalmente el criterio.

20.5 Una clave que existe pero no está autorizada

El caso `soloauth` tiene la clave solo en `authentication` (su `assertionMethod` está vacío). Se firma con ella y se verifica con cada propósito:

```
$ JWSS=$(TOOLS sign --key c007-clave.json --kid "did:web:evil.lab:casos:soloauth#key-1" --message "x"
| tail -1); echo "-- como firma de ASERCIÓN (assertionMethod) --"; VERIFICAR "$JWSS" --purpose
assertionMethod; echo "-- como AUTENTICACIÓN (authentication) --"; VERIFICAR "$JWSS" --purpose
authentication
-- como firma de ASERCIÓN (assertionMethod) --
RECHAZADA KEY_NOT_AUTHORIZED: La clave did:web:evil.lab:casos:soloauth#key-1 no está autorizada para
```

```
assertionMethod
```

```
-- como AUTENTICACIÓN (authentication) --  
VALIDA did=did:web:evil.lab:casos:soloauth kid=did:web:evil.lab:casos:soloauth#key-1 payload=x
```

- Como **aserción**: `KEY_NOT_AUTHORIZED` (la clave existe pero no está para eso).
- Como **autenticación**: `VALIDA`.

Así queda demostrado que **existir en el documento no basta**: la relación importa.

Conclusión — evidencia del criterio 3

- **DID inexistente** → `notFound` / `RESOLUTION_FAILED`.
- **id desajustado** → `invalidDidDocument` · `ID_MISMATCH`.
- **Documento no conforme** → `invalidDidDocument` con el código del defecto (contexto, secreto, referencia, clave).
- **Hostilidad de red** → redirección, tipo de contenido, tamaño: rechazados.
- **Desactivado** → `DEACTIVATED`.

21. CRITERIO 4 — El consumidor no realiza operaciones de escritura ni de publicación

Criterio. "El consumidor no realiza operaciones de escritura ni de publicación; evidencia: revisión de interfaz y pruebas de solo lectura."

Qué se busca. Cuatro evidencias independientes: (1) la **interfaz** no ofrece escritura; (2) las **dependencias** no permiten escribir; (3) el **código** no contiene llamadas de escritura; (4) en la **práctica**, un servidor espía solo vio `GET`.

Predice antes de ejecutar

El servidor `evil-lab` registró todas las peticiones del consumidor durante el laboratorio D.
¿Qué métodos HTTP esperas ver?

21.1 Revisión de la interfaz

```
$ cd /home/geovani/Descargas/generic/bitacora/new_proyect  
$ sed -n '/^\/* Contrato del consumidor/,/^}/p' did-resolver/src/main/kotlin/co/org/avance/ssi/resolver/Resolver.kt  
/** Contrato del consumidor: solo LEER. No hay operaciones de escritura ni de publicación en esta  
interfaz (ERSo 007, criterio 4). */  
interface DidResolver {  
    /** Resolver: DID -> DID Document. */  
    suspend fun resolve(did: String): ResolutionResult  
  
    /** Dereferenciar: DID URL (con fragmento) -> recurso concreto dentro del documento (p. ej. un  
método de verificación). */  
    suspend fun dereference(didUrl: String): DereferenceResult  
}
```

El contrato `DidResolver` ofrece **dos** operaciones, ambas de lectura (`resolve`, `dereference`). No existe ninguna `publish`, `write`, `put`, `update`...

21.2 Revisión de dependencias

```
$ sed -n '/^\// ERSo/,/^}/p' did-resolver/build.gradle.kts
// ERSo 2026-007: el consumidor NO depende de vdr-service (ni de nada que sepa escribir).
dependencies {
    api(project(":did-core"))
    api(libs.ktor.client.core)
    implementation(libs.ktor.client.cio)
    implementation(libs.kotlinx.coroutines.core)
    testImplementation(kotlin("test"))
    testImplementation(libs.ktor.client.mock)
    testImplementation(libs.kotlinx.coroutines.core)
}
$ ./gradlew :did-resolver:dependencies --configuration runtimeClasspath -q 2>&1 | grep -E 'project
:|vdr' | sort -u
+--- project :did-core
```

- La primera salida es el bloque de dependencias del módulo; el comentario explica la regla. **No figura `vdr-service`**.
- La segunda lista, de las dependencias del módulo en tiempo de ejecución, solo un proyecto interno: `did-core` (utilidades de DID y JWS). El consumidor **no tiene en su código** el servicio de registro.

21.3 Revisión del código

```
$ grep -rnE '\.(put|post|delete|patch|submitForm)\(|HttpMethod\.(Put|Post|Delete|Patch)' did-
resolver/src/main || echo '(ninguna coincidencia: el módulo no contiene llamadas de escritura)'; grep
-rn 'http.get' did-resolver/src/main | cut -c1-150
(ninguna coincidencia: el módulo no contiene llamadas de escritura)
did-resolver/src/main/kotlin/co/org/avance/ssi/resolver/Resolver.kt:73:             http.get(url) {
header(HttpHeaders.Accept, "application/did+json, a
```

- **Qué hace.** Busca en el código del módulo cualquier llamada que no sea de lectura: `put`, `post`, `delete`, `patch`, `submitForm`, o los métodos `Put/Post/Delete/Patch`.
- **Qué significa. Ninguna coincidencia.** La única petición que hace el módulo es `http.get(url)` (línea 73 de `Resolver.kt`).

21.4 La prueba en la práctica: lo que vio un servidor espía

```
$ echo "métodos HTTP que el servidor malicioso recibió del consumidor (todos los casos de $D):";
docker logs evil-lab 2>&1 | grep -E "^[0-9.]+" - - \[" | awk '{print $6}' | tr -d '"' | sort | uniq -
c; echo; docker logs evil-lab 2>&1 | grep -E "^[0-9.]+" - - \[" | sed -E 's/^[0-9.]+" - - \[["]*\]' //
| cut -c1-70 | head -20
métodos HTTP que el servidor malicioso recibió del consumidor (todos los casos de $D):
16 GET

"GET /casos/bueno/did.json HTTP/1.1" 200 532 "-" "ktor-client" "-"
"GET /casos/idajeno/did.json HTTP/1.1" 200 539 "-" "ktor-client" "-"
"GET /casos/sinctx/did.json HTTP/1.1" 200 436 "-" "ktor-client" "-"
"GET /casos/privada/did.json HTTP/1.1" 200 580 "-" "ktor-client" "-"
"GET /casos/dangling/did.json HTTP/1.1" 200 547 "-" "ktor-client" "-"
"GET /casos/tipo/did.json HTTP/1.1" 200 501 "-" "ktor-client" "-"
"GET /casos/notjson/did.json HTTP/1.1" 200 16 "-" "ktor-client" "-"
"GET /casos/html/did.json HTTP/1.1" 200 527 "-" "ktor-client" "-"
"GET /casos/grande/did.json HTTP/1.1" 200 200554 "-" "ktor-client" "-"
```

```
"GET /casos/redirect/did.json HTTP/1.1" 302 145 "-" "ktor-client" "-"
"GET /casos/gone/did.json HTTP/1.1" 410 143 "-" "ktor-client" "-"
"GET /casos/noexiste/did.json HTTP/1.1" 404 153 "-" "ktor-client" "-"
"GET /casos/sooauth/did.json HTTP/1.1" 200 500 "-" "ktor-client" "-"
"GET /casos/sooauth/did.json HTTP/1.1" 200 500 "-" "ktor-client" "-"
"GET /casos/sooauth/did.json HTTP/1.1" 200 500 "-" "ktor-client" "-"
"GET /casos/sooauth/did.json HTTP/1.1" 200 500 "-" "ktor-client" "-"
```

- **Qué hace.** Muestra los métodos HTTP que recibió `evil-lab` (el registro de acceso de nginx), contados.
- **Qué significa. 16 peticiones, todas GET.** Ni un `POST`, `PUT` o `DELETE` durante toda la batería de casos. (La línea `redirect` muestra además un `302` que el consumidor **no** siguió: no hay una segunda petición a la dirección de destino.)

🎯 Conclusión — evidencia del criterio 4

Evidencia	Resultado
Interfaz	Solo <code>resolve</code> y <code>dereference</code>
Dependencias	No depende de <code>vdr-service</code>
Código	Cero llamadas de escritura; solo <code>http.get</code>
Tráfico real	Solo <code>GET</code> observado por un servidor externo
Pruebas automáticas	«el consumidor solo emite GET» y «la interfaz pública no expone operaciones de escritura» (§22)

22. Pruebas automáticas y recorrido de extremo a extremo

22.1 Ejecutar las pruebas

```
$ ./gradlew :did-resolver:test --rerun-tasks -q 2>&1 | tail -3; echo '(sin salida = todas pasaron)'
(sin salida = todas pasaron)
$ TEST_DB_URL=jdbc:postgresql://127.0.0.1:55432/wallet_test ./gradlew :vdr-service:test --tests
'*ConsumerInterop*' --rerun-tasks -q 2>&1 | tail -3; echo '(sin salida = todas pasaron)'
(sin salida = todas pasaron)
$ python3 - <<'EOF'
> import re, glob, html
> for pat in ('did-resolver/build/test-results/test/*ResolverTest*.xml', 'vdr-service/build/test-
results/test/*ConsumerInterop*.xml'):
>     f = glob.glob(pat)[0]
>     s = open(f, encoding='utf-8').read()
>     tot = re.search(r'name="([^\"]+)" tests="(\d+)" skipped="(\d+)" failures="(\d+)"', s).groups()
>     print(f"{tot[0].split('.')[1]}: {tot[1]} pruebas, {tot[2]} omitidas, {tot[3]} fallos")
>     for m in re.finditer(r'<testcase name="([^\"]+)"[^\>]*?(/>|>(.*)</testcase>)', s, re.S):
>         name = html.unescape(m.group(1)).removesuffix("(")
>         estado = "FALLA" if "<failure" in (m.group(3) or "") else "PASA "
>         print(f" [{estado}] {name}")
>     print()
> EOF
ResolverTest: 12 pruebas, 0 omitidas, 0 fallos
[PASA ] CRITERIO 3 - did inexistente, invalido o desactivado se rechaza
[PASA ] clave que no existe o no esta autorizada para la relacion se rechaza
[PASA ] CRITERIO 4 - el consumidor solo emite GET
[PASA ] CRITERIO 2 - prueba firmada por clave ajena se rechaza
[PASA ] resuelve un did web por https y devuelve metadatos
[PASA ] pruebas malformadas se rechazan
[PASA ] no sigue redirecciones ni acepta content-type ajeno ni documentos gigantes
```

```
[PASA ] guarda - todo metodo de prueba devuelve void para que JUnit lo ejecute
[PASA ] dereferenciar devuelve el metodo de verificacion concreto
[PASA ] CRITERIO 3 - documento con id desajustado se rechaza
[PASA ] CRITERIO 1 - prueba firmada por la clave resuelta se valida
[PASA ] CRITERIO 4 - la interfaz publica no expone operaciones de escritura
```

```
ConsumerInteropTest: 1 pruebas, 0 omitidas, 0 fallos
```

```
[PASA ] el consumidor resuelve y verifica lo publicado, rechaza clave ajena y respeta rotacion y desactivacion
```

- **Qué hace.** La primera orden ejecuta las pruebas del módulo `did-resolver` (sin base de datos: usan un cliente HTTP simulado que **registra** los métodos usados). La segunda ejecuta la **interoperabilidad** con el registro real (necesita la base desechable `vdr-test-pg`).
- **Qué significa.** 12 + 1 pruebas, sin fallos. Los nombres con `CRITERIO n` indican qué criterio cubren.

Prueba	Criterio
CRITERIO 1 - prueba firmada por la clave resuelta se valida	1
CRITERIO 2 - prueba firmada por clave ajena se rechaza	2
clave que no existe o no esta autorizada para la relacion se rechaza	2
pruebas malformadas se rechazan	2
CRITERIO 3 - documento con id desajustado se rechaza	3
CRITERIO 3 - did inexistente, invalido o desactivado se rechaza	3
no sigue redirecciones ni acepta content-type ajeno ni documentos gigantes	3
CRITERIO 4 - el consumidor solo emite GET	4
CRITERIO 4 - la interfaz publica no expone operaciones de escritura	4
resuelve un did web por https y devuelve metadatos / dereferenciar devuelve...	pasos 1 y 3
el consumidor resuelve y verifica lo publicado, rechaza clave ajena y respeta rotacion y desactivacion (interop)	1, 2, 3

Preguntas (con respuesta)

1. ¿Por qué una prueba usa un «cliente simulado» para el criterio 4? → Porque así se puede **registrar** cada método HTTP que el código intenta usar, y comprobar que solo hay `GET`.
2. ¿Para qué sirve la prueba de «guarda»? → Garantiza que todo método de prueba devuelva `void`, porque JUnit no ejecuta los que devuelven valores y una prueba «silenciosa» daría falsa seguridad.

22.2 El recorrido de extremo a extremo (extracto del registro)

El script `scripts/e2e.sh` ejecuta todo el proyecto (121 comprobaciones). Este es el bloque de la ERSo 007:

```
ERSo 2026-007 · Resolución y verificación como consumidor conforme
```

```
- el consumidor resuelve el DID institucional publicado (solo lectura, GET por HTTPS) -
```

```

didDocument: {
  "@context": [
    "https://www.w3.org/ns/did/v1",
    "https://w3id.org/security/multikey/v1"
  ],
  "assertionMethod": [
    "did:web:civica-desarrollo.avance.org.co:entidades:avance#key-1"
  ],
  "authentication": [
    "did:web:civica-desarrollo.avance.org.co:entidades:avance#key-1"
  ],
  "id": "did:web:civica-desarrollo.avance.org.co:entidades:avance",
  "service": [
    {
      "id": "did:web:civica-desarrollo.avance.org.co:entidades:avance#issuer",
      "serviceEndpoint": "https://civica-desarrollo.avance.org.co/issuer",
      "type": "OID4VCI"
    }
  ],
  "verificationMethod": [
    {
      "controller": "did:web:civica-desarrollo.avance.org.co:entidades:avance",
      "id": "did:web:civica-desarrollo.avance.org.co:entidades:avance#key-1",
      "publicKeyMultibase": "zDnaegWdP7UT2CWtpx8dk5S8EtggrMUuN6dKmPhXdmHGZgnsE",
      "type": "Multikey"
    }
  ]
}
resolutionMetadata: ResolutionMetadata(error=null, message=null,
contentType=application/did+json, url=https://civica-desarrollo.avance.org.co/entidades/avance/did...
documentMetadata: DocumentMetadata(deactivated=false, versionId=1,
contentHash=sha256:2ce8444ae71f931f273979e362cdf505baca26b96ce88a4f0b5a3879e6cb8f3c)
VALIDA did=did:web:civica-desarrollo.avance.org.co:entidades:avance kid=did:web:civica-
desarrollo.avance.org.co:entidades:avance#key-1 payload=solicitud-de-presenta...
✓ PASS C1: prueba firmada por la clave resuelta -> VÁLIDA

RECHAZADA INVALID_SIGNATURE: La firma no corresponde a la clave publicada
✓ PASS C2: prueba firmada por una clave ajena -> RECHAZADA (INVALID_SIGNATURE)

RECHAZADA DID_MISMATCH: La prueba fue firmada por did:web:civica-
desarrollo.avance.org.co:entidades:avance y se esperaba did:web:civica-desarrollo.avance.org.co:ent...
✓ PASS C3: DID que no coincide con el de la prueba -> RECHAZADA (DID_MISMATCH)

RECHAZADA RESOLUTION_FAILED: notFound: El DID no existe en https://civica-
desarrollo.avance.org.co/entidades/fantasma/did.json
✓ PASS C3: DID inexistente -> RECHAZADA (RESOLUTION_FAILED / notFound)

RECHAZADA DEACTIVATED: El DID está desactivado
✓ PASS C3: DID desactivado -> RECHAZADA (DEACTIVATED)
✓ PASS C4: el consumidor solo usa GET (las pruebas unitarias con MockEngine y de interfaz lo
verifican)

```

22.3 Limpieza

El servidor malicioso es **desechable**; se elimina al final:

```

$ cd /home/geovani/Descargas/generic/bitacora/new_proyect/deploy; docker rm -f evil-lab; rm -rf
../evidencias/work/evil007
evil-lab

```

PARTE V — PREGUNTAS Y RESPUESTAS

23. Preguntas sobre los conceptos

- 1. ¿Qué diferencia hay entre resolver y dereferenciar?** *Resolver*: del DID (`did:web:...:avance`) al **documento completo**. *Dereferenciar*: de una DID URL con fragmento (`...:avance#key-1`) a **un elemento concreto dentro del documento** (esa clave o ese servicio). Están implementadas como dos operaciones separadas.
- 2. ¿Por qué no basta con que la firma sea matemáticamente correcta?** Porque una firma correcta solo demuestra que *alguien con esa clave* firmó. Hay que comprobar además que la clave **pertenece al DID, está en el documento y está autorizada para el propósito**. Sin eso, un atacante podría firmar con una clave propia y declarar que es de otra entidad.
- 3. ¿Qué diferencia hay entre `authentication` y `assertionMethod`?** La primera sirve para demostrar identidad (responder un desafío); la segunda, para afirmar cosas (emitir/firmar credenciales). Una clave puede estar en una, en otra o en ambas. Verificar contra la relación equivocada se rechaza (`KEY_NOT_AUTHORIZED`).
- 4. ¿Qué es el `kid` y por qué es una DID URL?** Es el identificador de la clave que dice haber firmado. Al ser `did:web:...#key-1`, contiene el DID (para saber qué documento resolver) y el fragmento (para saber qué clave buscar dentro).
- 5. ¿Por qué el `id` del documento debe coincidir con el DID pedido?** Para impedir que un servidor entregue el documento de **otra** entidad y lo haga pasar por el solicitado. Sin la comprobación, quien controle un servidor podría suplantar identidades.
- 6. ¿Por qué el consumidor no sigue redirecciones?** Una redirección permite que el servidor mande al cliente a otro lugar (posiblemente controlado por un atacante, o con otro dominio). Un DID debe resolverse **donde su regla dice**, no donde alguien indique.
- 7. ¿Por qué hay un límite de 128 KiB?** Para que un servidor hostil no pueda agotar la memoria del consumidor con un documento gigante. Un DID Document legítimo pesa unos cientos de bytes.
- 8. ¿Qué pasa con una firma hecha con una clave que luego se rotó?** Se rechaza (`INVALID_SIGNATURE`): el consumidor valida contra el documento **actual**. Es una limitación conocida (§27).
- 9. ¿Por qué el contenido de `contentHash` lo calcula el consumidor y no se lee del servidor?** Porque el servidor podría mentir. El hash propio permite a quien llama detectar cambios y construir una caché fiable.
- 10. ¿Por qué un DID desactivado no devuelve error sino `deactivated=true`?** Así lo define DID Resolution: «desactivado» es un **estado legítimo** del documento, no un fallo técnico. Pero el verificador de pruebas lo trata como rechazo (`DEACTIVATED`).

11. ¿Qué significa «consumidor conforme»? Que sigue las reglas del estándar y rechaza lo que no las cumple: no es «tolerante» con documentos defectuosos.

12. ¿Por qué `vdr-service` solo depende de `did-resolver` en pruebas? Porque el registro (productor) no necesita resolver para publicar. Mantenerlos separados evita acoplar las dos responsabilidades.

24. Preguntas por laboratorio y por criterio

Pasos 1 y 2 (laboratorio A)

P1. ¿Qué esperas en `error`, `violations` y `versionId`? ¿De dónde sale `versionId`? → `error=null`, `violations=[]`, `versionId=1`. Sale del encabezado `ETag` que entrega el servidor.

P2. ¿Qué hace el consumidor con el DID antes de pedir nada? → Aplica la regla `did:web` para calcular la URL (`https://<dominio>/<ruta>/did.json`). No hay directorio que consultar.

Criterio 1 (laboratorio B)

P3. ¿Qué pasa si verifico la prueba para cada propósito? → `VALIDA` en ambos, porque la clave está en `authentication` y en `assertionMethod`.

P4. ¿Qué dice la cabecera de la prueba y por qué no basta? → El algoritmo y el `kid`. Es una afirmación del firmante; el consumidor debe comprobarla con la clave publicada.

Criterio 2 (laboratorio C)

P5. Prueba con clave del atacante y `kid` de Avance, ¿qué código? → `INVALID_SIGNATURE`.

P6. Y si alteran el mensaje de una prueba legítima? → `INVALID_SIGNATURE`; la firma protege el contenido.

P7. ¿Por qué `KEY_NOT_FOUND` aparece antes de verificar la firma? → Porque sin la clave publicada no hay contra qué comparar.

P8. ¿En qué se diferencia `DID_MISMATCH` de `INVALID_SIGNATURE`? → `DID_MISMATCH`: la prueba es de otro DID distinto del que **yo esperaba**; la firma puede ser perfecta. `INVALID_SIGNATURE`: el DID es el esperado pero la firma no corresponde.

P9. ¿Qué muestra la rotación? → Que lo firmado con la clave retirada deja de valer; solo valen las firmas con la clave vigente.

Criterio 3 (laboratorio D)

P10. Documento perfecto pero con `id` de otro DID, ¿se acepta? → No: `invalidDidDocument` con `ID_MISMATCH`.

P11. ¿Qué códigos distintos aparecen para «inexistente», «desactivado» y «mal formado»?

Situación	Resolución	Verificación de prueba
Inexistente	<code>notFound</code>	<code>RESOLUTION_FAILED</code>

Situación	Resolución	Verificación de prueba
Desactivado	deactivated=true (sin error)	DEACTIVATED
Mal formado	invalidDidDocument + violaciones	RESOLUTION_FAILED
Tipo de contenido ajeno	representationNotSupported	RESOLUTION_FAILED
Demasiado grande	invalidDidDocumentLength	RESOLUTION_FAILED
Redirección	internalError	RESOLUTION_FAILED

P12. *¿Por qué hizo falta un servidor malicioso propio?* → Porque el registro real **no deja publicar** documentos defectuosos. Para probar al consumidor contra ellos hace falta un servidor que no filtre.

P13. *¿Por qué el certificado de `evil.lab` lo firma nuestra CA?* → Para que la conexión TLS sea válida y se pruebe **el contenido**, no el canal. Si el certificado fuera inválido, el consumidor rechazaría la conexión antes de leer nada, y no se probaría lo que se quiere probar.

P14. `soloauth`: *¿qué prueba?* → Que una clave presente en el documento pero **no listada** en la relación pedida se rechaza (`KEY_NOT_AUTHORIZED`).

Criterio 4 (laboratorio E)

P15. *¿Qué métodos HTTP vio el servidor espía?* → Solo `GET` (16 de 16).

P16. *¿Cuatro formas de demostrar «solo lectura»?* → Interfaz, dependencias, código, tráfico real (más las pruebas automáticas).

P17. *¿Qué evidencia se le da a quien objeta «¿y si mañana alguien añade un `put`?»?* → La prueba de reflexión («la interfaz pública no expone operaciones de escritura») fallaría y rompería la compilación continua; y el módulo no tiene dependencia del registro, por lo que no tendría cómo llamarlo.

Pruebas automáticas

P18. *¿Cuántas pruebas hay y cuál es la de interoperabilidad?* → 12 del módulo + 1 de interoperabilidad con el registro real (`ConsumerInteropTest`).

PARTE VI — CÓMO RESPONDER A CADA CRITERIO

25. Redacción modelo de las cuatro respuestas

Criterio 1 — Una prueba firmada por la clave resuelta se valida correctamente

El consumidor (DidWebResolver + ProofVerifier) resolvió `did:web:...:entidades:inst-007-...` por HTTPS, comprobó su conformidad y verificó una prueba JWS ES256 firmada con la clave privada correspondiente, con resultado `VALIDA` para `assertionMethod` y para `authentication`. Lo mismo con el DID institucional real `entidades:avance`. **Evidencia:** §17 y §18; prueba CRITERIO 1 - prueba firmada por la clave resuelta se valida.

Criterio 2 — Una prueba firmada por una clave ajena se rechaza

Una prueba que declara el `kid` del DID pero está firmada por otra clave se rechaza con `INVALID_SIGNATURE`; también una prueba con contenido alterado (`INVALID_SIGNATURE`), con clave inexistente (`KEY_NOT_FOUND`), de un DID distinto del esperado (`DID_MISMATCH`), mal formada (`MALFORMED_PROOF`) o hecha con una clave retirada por rotación (`INVALID_SIGNATURE`). **Evidencia:** §19; pruebas CRITERIO 2 ... y clave que no existe o no esta autorizada...

Criterio 3 — Documento no conforme, con `id` desajustado o inexistente, se rechaza

El consumidor rechaza: DID inexistente (`notFound` → `RESOLUTION_FAILED`), DID desactivado (`DEACTIVATED`), documento con `id` desajustado (`invalidDidDocument · ID_MISMATCH`), sin contexto (`INVALID_CONTEXT`), con material privado (`PRIVATE_KEY_MATERIAL`), con referencia colgante (`DANGLING_REFERENCE`), con clave inválida (`INVALID_KEY`), contenido no JSON, tipo de contenido ajeno, tamaño excesivo y redirecciones. También una clave presente pero no autorizada para el propósito (`KEY_NOT_AUTHORIZED`). Los documentos defectuosos se sirvieron desde un servidor de laboratorio (`evil.lab`) que no filtra. **Evidencia:** §20; pruebas CRITERIO 3 ... y no sigue redirecciones...

Criterio 4 — No realiza operaciones de escritura ni de publicación

La interfaz `DidResolver` solo ofrece `resolve` y `dereference`; el módulo `did-resolver` no depende de `vdr-service`; el código solo contiene `http.get`; y un servidor de laboratorio que recibió 16 peticiones del consumidor registró 16 `GET`. Las pruebas automáticas lo verifican con un cliente simulado y por reflexión. **Evidencia:** §21; pruebas CRITERIO 4 ...

26. Lista de comprobación para quien acepta

☐ Qué comprobar	Cómo	Resultado esperado
☐ Resolución	<code>tools resolve --did ...</code>	<code>error=null</code> , <code>violations=[]</code> , <code>versionId</code> , <code>contentHash</code>
☐ Prueba válida (aserción)	<code>sign + verify --purpose assertionMethod</code>	VALIDA
☐ Prueba válida (autenticación)	<code>verify --purpose authentication</code>	VALIDA
☐ Clave ajena	firmar con otra clave y mismo kid	INVALID_SIGNATURE
☐ Contenido alterado	cambiar el contenido	INVALID_SIGNATURE
☐ Clave inexistente	kid con <code>#key-9</code>	KEY_NOT_FOUND
☐ DID distinto al esperado	<code>--did</code> de otra entidad	DID_MISMATCH
☐ Prueba mal formada	<code>--jws</code> basura	MALFORMED_PROOF
☐ Clave no autorizada	caso <code>soloauth</code> con <code>assertionMethod</code>	KEY_NOT_AUTHORIZED
☐ DID inexistente	resolver un DID no publicado	<code>notFound</code> / <code>RESOLUTION_FAILED</code>
☐ DID desactivado	resolver tras <code>DEACTIVATE</code>	<code>deactivated=true</code> / <code>DEACTIVATED</code>
☐ id desajustado	caso <code>idajeno</code>	ID_MISMATCH
☐ Documento defectuoso	casos <code>sinctx</code> , <code>privada</code> , <code>dangling</code> , tipo	<code>invalidDidDocument</code> + código
☐ Defensas de red	casos <code>html</code> , <code>grande</code> , <code>redir</code>	rechazados
☐ Solo lectura (interfaz)	revisar <code>DidResolver</code>	solo <code>resolve</code> / <code>dereference</code>
☐ Solo lectura (dependencias)	revisar <code>build.gradle.kts</code>	sin <code>vdr-service</code>
☐ Solo lectura (tráfico)	registro de <code>evil-lab</code>	solo <code>GET</code>
☐ Pruebas automáticas	<code>./gradlew :did-resolver:test</code>	12/12

PARTE VII — LÍMITES, HALLAZGOS Y OPERACIÓN

27. Lo que este informe NO demuestra, y lo que se encontró

27.1 Límites

Límite	Detalle
Se valida contra el documento actual	La rotación de claves no concede validez histórica: una firma hecha con una clave ya rotada se rechaza (§19.6). Validar «con la clave de su época» requeriría el historial, que existe en el registro (ERSo 008) pero no se expone públicamente ni el consumidor lo usa. Es trabajo futuro.
Solo Multikey P-256	Otros tipos (JsonWebKey2020 , Ed25519...) se rechazan. Coherente con la ERSo 005; no cubre el ejemplo general del equipo.
Sin caché	El módulo expone <code>versionId</code> y <code>contentHash</code> para que quien lo use implemente caché, pero no trae una. La ERSo pide «revisar» las buenas prácticas, no implementarlas.
Formato de prueba propio	JWS compacto ES256 del proyecto, no una <i>Data Integrity Proof</i> de W3C.
Versiones normativas	Ver <code>docs/VERSIONES-NORMATIVAS.md</code> ; confirmar versión y estado de cada especificación antes de citarla formalmente.
Solo did:web	No se integra ION ni anclajes equivalentes (es la condición 1).
Sin juicio de identidad jurídica	Una firma válida no dice quién es la persona o entidad ante la ley; solo que la clave firmó.
Servidor malicioso de laboratorio	Prueba las defensas del cliente, pero no sustituye una auditoría de seguridad del resoledor (DNS, certificados revocados, etc.).
Comprobación de revocación de certificados	El consumidor confía en la CA de laboratorio; no consulta listas de revocación.

27.2 Hallazgo heredado: HEAD responde 404 en el registro

Descubierto en la ERSo 005. No afecta al consumidor (que solo usa `GET`), pero conviene corregirlo en el productor.

28. Si algo no sale como en el informe

Síntoma	Causa probable	Qué hacer
<code>resolve da internalError: No se pudo leer...</code>	El servicio no está arriba o la CA no se montó	<code>docker compose up -d</code> ; comprobar <code>./certs/ca.crt</code>

Síntoma	Causa probable	Qué hacer
evil.lab no resuelve	El contenedor evil-lab no está en la red vdr-ssi_default	Repetir §20.3
docker: network vdr-ssi_default not found	El proyecto se llama distinto	docker network ls ; ajustar el nombre en el comando
openssl x509 -req ... CA falla	ca.key no está en deploy/certs/	Regenerar con scripts/gen-dev-certs.sh
Todas las verificaciones dan RESOLUTION_FAILED	La clave y el DID no se publicaron	Repetir §16.1
412 NAMESPACE_NOT_OWNED al publicar	Falta el espacio entidades/* (ver informe de la ERSO 006, §12.5)	Reservarlo y reiniciar vdr
Las pruebas de interoperabilidad no conectan	Falta la base desechable	Levantar vdr-test-pg (puerto 55432)
Queda evil-lab corriendo	El laboratorio se interrumpió	docker rm -f evil-lab

29. Operación diaria

```

cd deploy
docker compose up -d           # levantar
docker compose --profile tools down # bajar (NUNCA con -v: borra la base)
bash scripts/lab-007-completo.sh # repetir todos los laboratorios

```

- **Usar el consumidor desde otro módulo:** añadir `implementation(project(":did-resolver"))` y crear `ProofVerifier(DidWebResolver())`.
- **Verificar credenciales:** el servicio `credential-service` ya usa `VerificationKeyResolver` para localizar la clave del emisor por su DID (ERSO 002).

ANEXOS

Anexo A — Los scripts

Script	Para qué sirve
scripts/lab-007-completo.sh	Ejecuta los laboratorios A a G, incluida la limpieza del servidor malicioso
scripts/e2e.sh	Recorrido de extremo a extremo (121 comprobaciones)
scripts/gen-dev-certs.sh	Genera la CA y los certificados de laboratorio
scripts/generar-pdf.sh	Convierte un informe a PDF

Anexo B — Autocomprobación (con respuestas)

1. ¿Qué códigos puede devolver `ProofVerifier` al rechazar? → `MALFORMED_PROOF`, `DID_MISMATCH`, `RESOLUTION_FAILED`, `DEACTIVATED`, `KEY_NOT_FOUND`, `KEY_NOT_AUTHORIZED`, `INVALID_KEY`, `INVALID_SIGNATURE`.
2. ¿Qué diferencia hay entre `KEY_NOT_FOUND` y `KEY_NOT_AUTHORIZED`? → La primera: la clave no está en el documento; la segunda: está, pero no en la relación pedida.
3. ¿Qué errores normativos de resolución existen? → `invalidDid`, `notFound`, `invalidDidDocument`, `representationNotSupported`, `invalidDidDocumentLength`, `internalError`.
4. ¿Por qué no se siguen redirecciones? → Para no ser desviados a un destino que el DID no dicta.
5. ¿Cuántas operaciones tiene la interfaz? → Dos: `resolve`, `dereference`.
6. ¿Dónde se usa `did-resolver` en producción? → En `credential-service`, `did-tools` y `wallet-sim`; en el registro solo en pruebas.
7. ¿Qué ocurre con una firma de una clave rotada? → `INVALID_SIGNATURE`.
8. ¿Qué prueba que el consumidor solo lee? → Interfaz, dependencias, código y tráfico observado.

Anexo C — Ejercicios del método (con respuesta modelo)

Ejercicio 1. Descompón el criterio 3. → Tres familias: documento **no conforme**, **id desajustado**, **inexistente**. Evidencia: prueba negativa de cada una. Mecanismos: validador `CONSUMER`, comparación de `id`, códigos `notFound` / `RESOLUTION_FAILED`.

Ejercicio 2. Diseña una prueba para «el consumidor no sigue redirecciones». → Servidor que responde `302` a un documento válido; el consumidor debe fallar con `internalError` y el servidor no debe registrar una segunda petición (es el caso `redir`).

Ejercicio 3. ¿Cómo demostrarías que un cambio futuro no rompe el criterio 4? → Las pruebas «solo emite GET» y «la interfaz no expone escritura» deben seguir pasando; y revisar que `build.gradle.kts` no incorpore `vdr-service`.

Anexo D — Referencias

- W3C — Decentralized Identifiers (DIDs) v1.1.
- W3C — DID Resolution v1.
- Especificación del método `did:web`.
- Multikey y multibase; RFC 7515 (JWS).
- `docs/VERSIONES-NORMATIVAS.md`, `docs/MARCO-CONCEPTUAL.md`, `docs/COMO-SE-RESOLVIO-CADA-CRITERIO.md`.